Supplementary Material: Unused Mantissa Bits on Power-Capped GPUs

Anonymous

This supplement collects measurements that support, but are not needed for, the conclusions of the paper: results obtained through the GH200 interposer (upper bounds), third-party non-GEMM codes, single-precision replacements, the GH200 emulation shape sweep, further Gaussian-process experiments, the GH200 HPL tuning, and the AI workloads. Section references of the form "Section III" point to the paper.

S1. Interposer Results on GH200 (Upper Bounds)

Plane-wave DFT is tolerant. With 28 kept bits, the total energies of QE Si 1000 and Au 112 move by 1.0×10^{-5} and 2.3×10^{-5} Ry under truncation and by 3.5×10^{-7} and 2.8×10^{-7} Ry under round-to-nearest. The gold surface still converges at 20 bits with nearest rounding (5.5×10^{-5} Ry, both runs); for Si 1000, 20 bits is the failure edge (one run converges with 1.1×10^{-4} Ry, the other aborts at iteration 24), and truncation diverges or slows at 20 bits. Gaussian-basis quantum chemistry needs more bits. GPU4PySCF valinomycin DFT errs by 1.1×10^{-8} E_h at 44 bits; at 36 bits the energy oscillates near 10^{-8} E_h and cannot meet a 10^{-10} convergence threshold, but with the chemically sufficient 10^{-7} threshold 40, 36 and 32 bits converge in the same 12 iterations as native, with a 32-bit nearest error of 1.2×10^{-5} E_h (≈0.007 kcal/mol). At 36–40 bits the error is then set mainly by the threshold (32 bits errs an order of magnitude more), consistent with the ≈37 significant bits found sufficient by Dawson et al. [1].

Under caps, with the shim truncating every GEMM and compared against a baseline that takes the same copy path without truncating, Si 1000 at 550 W with 28-bit nearest rounding is 6.2% faster and uses 6.4% less energy (SM clock 1348→1446 MHz), and Au 112 at 450 W is 4.9% faster with 5.1% less energy (one truncated run each, between baseline runs before and after). These are upper bounds for a library that truncated internally; against the unmodified program the shim is 0.4% slower (Au 112, 450 W) because of its ≈5% copy cost. The in-program version of Section VI-C truncates only Davidson blocks without copies and is the deployable number.

For GPU4PySCF, nsys shows FP64 GEMM kernels at 46% of GPU kernel time and 32% of total runtime for valinomycin DFT (31% and 27% of kernel time for taxol and tamoxifen); the rest is dominated by copies of computed intermediates (36–50%) and integrals. With 32–36 bits needed and FP64 GEMM keeping 35 bits 5.1–6.5% faster at 500–800 W, truncation alone is bounded at about 32% × 5–6.5% ≈ 1.6–2.1% end to end (3.9% at 400 W). This estimate assumes the default grid and GEMM time proportional to flops at 1000 W speed; on a finer grid (level 6), valinomycin measured at 450–550 W is 3.3–3.5% faster (Section VII-B). Neither number implies that truncation beats INT8 emulation here: the two dominant GEMM shapes favor different methods (Section VII-A), and choosing by shape bounds the gain at about 3.5% with cuBLAS emulation for the long-k shape, or 6.6% with the less accurate GEMMul8.

Wide range, partly power-bound DFT. These DFT runs draw only 650–690 W at 1000 W, but are power-bound below that. For valinomycin (grid level 6) at 550 and 450 W, rounding to 36 bits through the shim is 3.3–3.5% faster than the copy baseline with 2–3% less energy and an error of 9.5×10^{-7} E_h; emulation must keep 40 bits to stay correct, which is then about ten times more accurate (8.4×10^{-8} E_h) but takes 1.48–1.53× the baseline time (two rounds each).

ExaGeoStat itself (GH200). We also built ExaGeoStatCPP (StarPU and Chameleon tile Cholesky with cuBLAS 13.6; tile size 1920) and ran its MLE (N = 60000, six BOBYQA likelihood evaluations per run, data generated once natively and read by every run) with the shim on its FP64 tile GEMMs. Matérn covariance generation runs on the CPU and takes about 9 s per evaluation against 2.3 s of GPU factorization at 1000 W, and the factorization is partly power-limited even there (GPU power about 620–670 W on average, frequent power-cap events). Relative to a baseline that copies the operands without truncating, the factorization is 3.0% (36 bits) and 4.0% (28 bits) faster at 1000 W, 5.0% and 8.0% at 600 W, and 5.6% and 9.4% at 450 W (means of two ABBA rounds; at 450 W the rounds differ widely, +6.9 and +11.9% with 28 bits at 707 and 926 MHz, because the CPU shares the module cap while generating the covariance); with 28 bits the mean SM clock during factorization rises from 1913 to 1975 MHz at 1000 W and from 1166 to 1305 MHz at 600 W. The log-likelihoods change by at most 1.2×10^{-10} and 9.8×10^{-8} relative. The shim's own copy costs 7–9%, which leaves it 0.5–3.1% slower than unmodified ExaGeoStat, so the gain needs an implementation inside the library. ExaGeoStatCPP's mixed-precision option (`--computation=mixed`) is only an enumeration value without an implementation, so we could not compare with its own mixed precision.

S2. Third-Party Non-GEMM Codes (GH200)

Third-party codes (GH200). We zeroed, once at setup, the coefficient arrays that are stored and multiplied in every step in three GPU FP64 codes: MFEM's partial-assembly quadrature data (about 43% of CG memory traffic), SW4's material parameters (about 9%) and LAMMPS EAM tables (cache

resident). Without zeroing all three are bitwise identical to the originals or within rerun noise. MFEM (38.27 M unknowns, $p = 3$) is 0.9/2.0% faster with 36/20 bits (nearest) at 450 W and 0.7/1.4% at 550 W, unchanged at 1000 W; the solution changes by at most 2.3×10^{-11}. SW4 (123 M grid points) is 0.6/1.3% faster at 350 W and 0.3/0.5% at 450 W, with receiver waveforms changing by at most $3.4\times10^{-11}/3.4\times10^{-6}$ (doubling the grid changes them by 4–10%); LAMMPS EAM (16.38 M atoms) moves 0.0–0.4%, within noise. The gain order, MFEM ≥ SW4 > LAMMPS, follows the coefficients' share of memory traffic: kernel-level gains of 4.8–14.0% at 450–500 W are diluted by the rest of the full-mantissa computation. Zeroing fits best when inputs flow directly and in volume into multipliers or data paths.

S3. FP32 and 3xTF32: Accumulation, Not Range (GH200)

3xTF32 (our own implementation, in the shim and in a benchmark script, not the CUTLASS one) is the fastest method in Table S2 but has only FP32-level accuracy, and its error grows with the inner dimension: on 4096×4096 it is 6.4×10^{-7}, 4.8×10^{-6} and 3.8×10^{-5} at $k = 256$, 2048 and 16384, whereas FP32 SGEMM grows as $\sqrt{k}$ (2.9×10^{-7}, 8.1×10^{-7}, 2.3×10^{-6}) and FP64 truncation is independent of k. The measured log slope of about one (7.5–7.9× per 8× in k) points to a systematic error in TF32 tensor-core accumulation rather than random rounding, consistent with the round-toward-zero accumulation Fasi et al. found on V100, T4 and A100 [2] (bit-accurate Hopper models: [3]). At $k = 2048$ its error is 18 times that of FP64 rounded to nearest at 20 bits (2.6×10^{-7}) and 7 times that of FP64 truncated to 20 bits (6.7×10^{-7}).

Quantum ESPRESSO's operands fit in FP32's range, so we replaced every FP64 and complex FP64 GEMM with single precision through the shim (inputs to FP32; 3xTF32 additionally splits into big and small parts multiplied three times on TF32 tensor cores; C combined in FP64). On the gold surface at 450 W (2 interleaved runs each), FP32 aborts at SCF iteration 26 and 3xTF32 and TF32 at iteration 2 when the Davidson bands fail to converge, and the integrated charge drifts from 1232 electrons (FP32 +0.012, 3xTF32 +0.074); FP64 rounded to 24 and 20 bits differs from native by 8.1×10^{-6} and 5.5×10^{-5} Ry and converges in 28 iterations. Si 1000 at 550 W behaves the same: FP32 aborts at iteration 14 and 3xTF32 at 9, with 4000.18 and 4001.52 electrons instead of 4000. The difference is accumulation: truncated FP64 still accumulates in FP64, while Quantum ESPRESSO's inner dimension is the number of plane waves (95386 on the gold surface plus a real GEMM with $k = 4.3\times10^6$; 375286 for Si 1000), where 3xTF32 already errs by 5.6×10^{-5} to 2.2×10^{-4}, the same order as plain TF32. FP32 GEMM on Hopper is also slower than FP64 tensor cores (on the gold surface about 1.5× per SCF iteration, estimated). Replacing every GEMM with single precision thus fails for this long-inner-product solver even though the range fits; mixed precision that uses FP32 only in selected steps, as Woo et al. do [4], is a different matter. FP64 truncation is the reduced-precision choice here that leaves the algorithm unchanged.

TABLE S1
GH200: the two GEMM shapes that each carry 35% of valinomycin DFT flops: tall-skinny 757248×300×2958 and long-k 2958×2958×76800 (TFLOP/s; relative error in parentheses; GEMMul8 errs by 1.0×10^{-10} and 8.2×10^{-10} on the two shapes, 10–80 times more than truncation to 36 bits).

	Tall-skinny		Long k	
Method	1000 W	575 W	1000 W	575 W
FP64 native	51.6	36.8	50.6	36.2
Truncation, 36 bits (1.0×10^{-11})	54.0	38.9	53.7	38.0
cuBLAS emu., 32 bits (9.7×10^{-12})	19.5	16.6	65.4	46.0
cuBLAS emu., 28 bits (2.3×10^{-9})	24.2	20.9	89.9	65.6
GEMMul8, 10 moduli	17.9	16.7	102	75.6

S4. Emulation Crossover on GH200: Shapes, Caps and a Model

The crossover depends on shape. Intercepting all 45,574 GEMMs of the valinomycin DFT run on GH200 reveals two shapes that each carry 35% of the flops (Table S1). With $n = 300$, emulation cannot amortize its splitting and scaling: even at 28 bits it runs at 0.47–0.57× native speed, and truncation wins at every accuracy. With $k = 76800$ the fixed costs amortize: emulation is still 21–22% faster than truncation at the 10^{-11} level, and GEMMul8 is nearly twice as fast at 10^{-9}. Within one application the best choice is per call: truncation for tall-skinny shapes, emulation for long k.

A shape sweep and a predictive model (GH200). We fixed $m = 8192$ and swept $n \in \{256, 1024, 4096, 8192\}$ and $k \in \{256, 1024, 4096, 16384, 81920\}$ at 1000, 575 and 450 W, two rounds each (480 records; Fig. S1). Truncation to 36 bits beat native in all 60 shape–cap combinations (3.8–11.9% at 450 W, 3.6–5.9% at 575 W, 0.5–4.8% at 1000 W). cuBLAS emulation with 32 bits wins roughly when $n \geq 4096$ and $k \geq 16384$ (and at $n = 1024$, $k = 81920$ at 1000 and 575 W) (at best 0.74× truncation time at 1000 W), and its winning region shrinks at lower caps: 5 of 20 shapes at 1000 and 575 W, 2 at 450 W. Where the emulation itself is not power-bound, however, lower caps favor it: at $n = 8192$, $k = 4096$ its time relative to truncation falls from 1.17 to 1.08 to 1.05 as the cap goes from 1000 to 575 to 450 W. The four values of n were measured on different nodes: each ratio is paired on one node, but shape effects and node differences cannot be separated. GEMMul8 with 10 moduli needs only 0.47–0.80× the truncation time when $n, k \geq 4096$, at an order of magnitude larger error ($\approx 10^{-10}$ versus 10^{-11}), and is up to 4.5× slower when n or k is small. Emulation's extra work is input splitting ($\propto mk + nk$) and output accumulation ($\propto mn$); dividing by the mnk multiply gives

$$T_{\text{emu}}/T_{\text{trunc}} = A + B\,(1/m + 1/n) + C/k, \qquad (1)$$

where A is the ratio of per-flop speeds. Least-squares fits per cap (about 20 points for three parameters, so R^2 is only descriptive) reach $R^2 = 0.98$–0.99 for GEMMul8, versus 0.86–0.89 for a single effective size $1/(1/m + 1/n + 1/k)$; cuBLAS emulation reaches only 0.88–0.89. On the two held-out valinomycin shapes the model picks the winner in 8 of 8

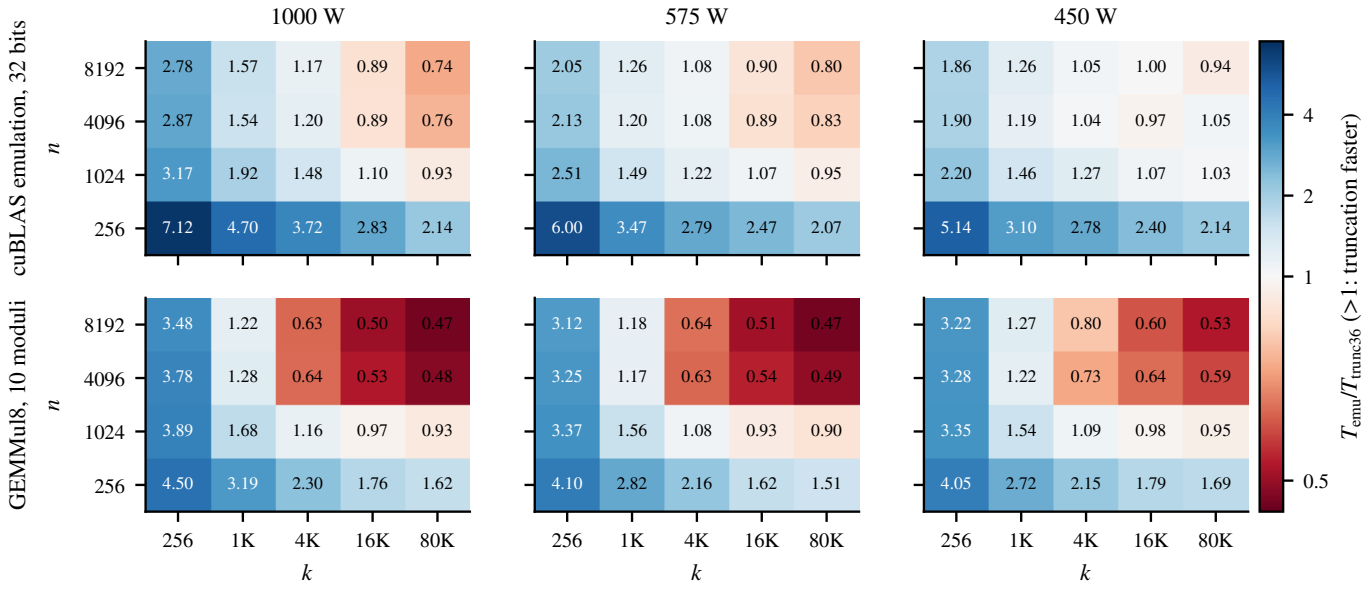


Fig. S1. GH200: time of INT8 emulation relative to truncation to 36 bits, $T_{emu}/T_{trunc36}$ (m = 8192; blue: truncation faster). Top: cuBLAS emulation with 32 bits (error comparable to truncation, $\approx 10^{-11}$); bottom: GEMMul8 with 10 moduli (about ten times larger error, $\approx 10^{-10}$); columns: module caps. Means of two rounds.

TABLE S2
GH200: SQUARE GEMM (32768×32768×2048) IN TFLOP/S BY CAP ON ONE NODE, NATIVE FP64 FIRST AND THE OTHER METHODS ORDERED BY RELATIVE ERROR.

Method (error)	1000 W	575 W	450 W
FP64 native	59.2	43.2	35.8
Trunc., 36 bits (1.0×10^{-11})	61.8	45.5	38.6
GEMMul8, 10 moduli (1.0×10^{-10})	70.8	51.4	34.7
GEMMul8, 9 moduli (1.6×10^{-9})	78.2	56.6	38.2
cuBLAS emu., 28 bits (2.3×10^{-9})	67.8	55.2	47.5
Trunc., 28 bits (2.6×10^{-9})	63.2	46.6	39.6
FP32 SGEMM (8.1×10^{-7})	51.2	40.4	34.6
3xTF32 (4.8×10^{-6})	96.6	72.0	56.4

cases, all far from the crossover, so this is a loose test; the model is an empirical description of these measurements.

The power of the emulation itself changes the winner. On GH200, GEMMul8 draws about 970 W at a 1000 W cap and at 450 W its clock drops to 650–720 MHz, where truncation becomes faster; cuBLAS fixed-point emulation with 28 bits draws only about 706 W at 1000 W and gains relative ground at lower caps on the square shape (Table S2). Which way a tighter cap tips the balance thus depends on how much power the emulation itself draws.

S5. Gaussian Process: Maximum-Likelihood Fit and Tile Mixed Precision (GH200)

A one-parameter maximum-likelihood fit (GH200). To check that the estimate itself is unchanged, we fitted the range parameter with variance and smoothness fixed: golden-section search over its logarithm, 17 likelihood evaluations per variant (n = 60000; 1000 and 700 W; two interleaved rounds). Truncation to 36 and 28 bits and cuBLAS emulation return the native estimate $\hat{\theta}$ = 0.05007952 in every printed digit, with maximized log-likelihoods within 4.8×10^{-6} (36 bits) and 2.3×10^{-4} (28 bits) of native; FP32 moves $\hat{\theta}$ by 0.49% and loses 0.72 in log-likelihood. At 700 W the fit is 7.1–7.3% faster with 28 bits and 4.8–4.9% faster with 36 bits, using 6.6–6.8% and 4.5–4.8% less energy; at 1000 W it is 1.8–2.3% and 1.6–2.2% faster with 9.1–9.9% and 3.5–5.6% less energy; cuBLAS emulation is 32–41% slower. Of the three whole-matrix choices we tested (FP32, 36-bit cuBLAS emulation and truncation), only truncation is both fast and accurate on this wide-range, power-bound workload.

Composing with tile mixed precision (GH200). The mixed-precision Cholesky factorizations of the ExaGeoStat line change the format of whole tiles instead (Section II). To see whether truncation still helps there, we ordered the locations along a Morton curve and added such a variant to our kernel: in the trailing update, tiles below each column's FP64 extent, chosen by the tile-norm rule with a required accuracy of 10^{-9} as in [5], use FP16 inputs with FP32 accumulation on the tensor cores (52% of the tiles at range 0.02, 3% at 0.08). Over the six ranges (n = 60000, two ABBA rounds) it runs 1.60–1.63× faster than native FP64 at 1000, 700 and 500 W, with log-likelihood deviations up to 0.043. Truncating its remaining FP64 tiles to 28 bits makes it another 1.9% faster at 1000 W and 7.0% and 6.1% faster at 700 and 500 W, with an unchanged deviation, so truncation composes with format changes rather than competing with them. Zeroing the FP16 operands to 5 mantissa bits as well added nothing.

S6. HPL Tuning and Node Sets on GH200

GH200, 8 nodes. We first tuned native HPL at 1000 W. The configuration inherited from an H200 study (N = 262144, NB 1024, 2 × 4, default NCCL) gave 374.8–376.0 TFLOP/s;

TABLE S3

GH200: DELIVERED PERFORMANCE ON 8 NODES IN TFLOP/S, NODE SET A / B (MEANS PER SET). HPL: TUNED, $N = 286720$; HPL-MxP: $N = 491520$, REPORTABLE RATE INCLUDING REFINEMENT. THE 8 GPUS' FP64 TENSOR-CORE PEAK IS ABOUT 536 TFLOP/S (8×67 TFLOP/S [6]).

	HPL (FP64)		HPL-MxP (BF16 LU)	
Cap	Native	36 bits	Native	3 bits
400 W	225.0/228.0	249.9/249.2	2076/1922	2340/2128
500 W	288.0/278.2	300.7/291.0	2823/2555	3007/2692
600 W	318.0/304.5	334.0/318.3	3229/2858	3310/2922
700 W	346.2/328.3	362.9/341.3	3449/3039	3519/3094
800 W	370.8/345.8	385.6/356.3	3655/3178	3720/3229
1000 W	393.9/365.5	409.2/367.5	—*	—

Set A: work4–11 (HPL), work4–7 and 10–13 (HPL-MxP), 2 pairs per cap; set B: work3–8, 12, 13, 4 pairs. *Native HPL-MxP trips the chassis power brake at 1000 W on set A; set B was not run there.

TABLE S4

WIKITEXT-2 PERPLEXITY (SEQUENCE 2048) RELATIVE TO BF16 FOR THE LINEAR LAYERS' FORMATS; FP8 USES REAL FP8 TENSOR-CORE GEMMS, AND KEPT BITS ARE ROUNDED TO NEAREST (TIES TO EVEN).

Format	L2-7B	L3.1-8B	Qw2.5-7B	Mis-7B	OPT-6.7B	L2-70B
BF16, 4 b	+0.09	+0.19	+0.13	+0.09	+0.45	+0.10
BF16, 3 b	+0.44	+1.02	+0.65	+0.41	+2.00	+0.51
FP8 per tensor	+0.63	+0.95	+0.70	+0.45	+2.23	+0.48
FP8 per channel	+0.32	+0.88	+0.61	+0.34	+0.69	+0.42
FP8 p.c., 2 b	+3.2	+6.5	+4.0	+2.5	+7.0	+3.4
FP8, wts. 2 b	+1.9	+4.0	+2.6	+1.3	+2.8	—

Values in percent. b: kept bits; L: Llama, Qw: Qwen, Mis: Mistral; p.c.: per channel; wts.: weights only.

fixing NCCL at 8 channels with 2 MiB buffers (+4.5%) and raising N to 286720 (+2%) gave 390.2–398.8 TFLOP/s in three runs, and no neighboring setting of N, NB, split fraction or NCCL parameters beat this ≈2% run-to-run spread (24 settings, 26 runs; $f = 0.15$ and 4×2 grids were about 2 and 13% slower). We then swept the caps with this configuration on two node sets: set A (work4–11) with two ABBA pairs per cap, and set B (work3–8, 12 and 13), measured a day later after one uncounted warm-up run, with four pairs per cap. That day the second InfiniBand port of most nodes linked at only 2.5 Gb/s and NCCL and MPI routed part of the traffic over it (157 TFLOP/s on 8 nodes); pinned to the 200 Gb/s port, set B still reached only 365.5 TFLOP/s natively at 1000 W, 7% below set A, although its SM clock was higher (1961 versus 1936 MHz), so more of its time went to communication. Differences between the sets thus mix nodes and network state; every pair is matched within one job.

S7. AI WORKLOADS: TRAINING AND INFERENCE (GH200)

FP8 inference and its failures. FP8 training keeps high-precision master weights with per-tensor scaling, E4M3 for weights and activations and E5M2 for gradients [7]; Gupta et al. showed that rounding decides the fate of low-precision training [8]. The clearest FP8 failure is attention: FlashAttention-3's FP8 mode drops the VQA-a video-quality score of CogVideoX from 70.2 to 6.5 and of HunyuanVideo from 82.5 to 4.4, and 262k-context retrieval from 7.0 to 0.4, which SageAttention2 repairs inside FP8 by smoothing outliers and using a fixed scale for softmax probabilities [9]. MLPerf Inference closed division allows any pure, reproducible mathematical quantization but forbids pruning non-zero weights [10]; mantissa zeroing is the former.

MLPerf Training measures time to target quality (eval loss ≤ 0.925 on GovReport for Llama 2 70B LoRA); a faster step that needs more samples to converge loses.

Training: truncating the FP8 base weights is a negative result. We truncated only the frozen FP8 base weights of Llama 2 70B LoRA (once, after loading); activations, gradients, LoRA weights and updates and per-tensor scaling follow [7]. On 8 nodes with the same seed, native FP8 at 575 W reached the target after 3456 samples in 28.34 min. Keeping 2 bits made each 384 samples only 0.9–1.2% faster but needed 4224 samples (22% slower at 575 W, 19% at 800 W); 1 bit did not converge (it also lowers the largest representable E4M3 value from 448 to 384, compressing the top weights by about 14%). The eval-loss curve shifts up by 0.006–0.008, while the train-loss gap varies (with 2 bits −0.031 at the first logged step, then +0.003 to +0.017): truncation weakens the base model at load time, and a rank-16 adapter does not compensate. A second seed (5678, 800 W) agrees: native 3072 samples in 22.74 min, 2 bits 4224 samples in 31.56 min (+39%). Across two seeds and four pairs (one at 575 W, three at 800 W), the first evaluation is always 0.008–0.010 higher and convergence takes 2–3 more evaluation intervals. Native convergence differs by one interval between node sets (3072 versus 3456), so we read this only as evidence that FP8's 3 mantissa bits leave no room. The rounding mode shows no consistent order (at 800 W truncation and stochastic rounding converge at 4224 samples, nearest at 5376). Truncating only MLP layers needs 4224 samples, attention only 3840 (one interval, within node-set noise, but attention is only about 18% of the linear weights); the first-evaluation gaps of +0.0064 (MLP) and +0.0028 (attention) roughly add up to the +0.0086 of full truncation (their sum is 0.0092). A repeat of the attention-only run gave nearly the same first evaluation (0.9428 versus 0.9425) but converged one interval later (4224 versus 3840, missing the target by 10^{-5} at 3840), so the first-evaluation gap, not the convergence point, is the robust metric.

Inference: FP8 has already taken most of the benefit. BF16 rounded to nearest at 3 bits has as many mantissa bits as FP8 E4M3 and matches its accuracy on six language models (Table S4; with truncation, 4 bits are needed for a similar GEMM error, 4.35% on uniform data). We then compared the two where FP8 is known to fail: attention perplexity at 2k/8k context (BF16 3 bits +0.49/+1.06%, emulated FP8 attention +0.52/+1.08%), key retrieval at 8k–32k context on Llama-3.1-8B (both 100%), BERT-large SQuAD (all formats above 99.9% of FP32 F1) and CogVideoX-2B video generation (PSNR against native 18–38 dB for FP8 attention, 20–29 dB for BF16 3 bits). With proper scaling the two tie, and FP8 GEMMs are about 1.7–1.9× as fast (at 575 W, 1.67–1.72× BF16 with 3 bits and 1.80–1.87× native BF16). A naive

FP8 attention that does not scale the softmax probabilities fails as sequences grow (74% error at 4096 tokens), and on CogVideoX-2B (about 18k tokens) it changes the composition and adds noise (PSNR 10–12 dB, SSIM 0.07–0.66) where BF16 with 3 bits keeps 20–21 dB and SSIM 0.82–0.91. This is a range problem, not a mantissa problem: the smallest positive E4M3 value, about 2^{-9}, exceeds the mean attention probability over 18k tokens, while BF16 has 8 exponent bits; correct implementations already fix it inside FP8 [9].

Without switching to 2:4 structured sparsity or INT4 weights, zeroing on top of FP8 is the remaining way to go faster than FP8 on Hopper, which lacks FP4. On real layer-40 activations and weights of Llama2-70B (8192×57344×8192), keeping 2 bits on both sides is 4.4/3.2/5.0/8.2% faster at 1000/700/575/450 W, weights only 1.3–3.3%, but 70B perplexity rises by 3.4%. Per-layer sensitivity shows q/k/v as least sensitive but a small share of the compute (with grouped-query attention, q is about 7.8% and k and v about 1% each), and the MLP layers as most of the compute but more sensitive. FP8's advantage comes mainly from halving the data path: at 575 W BF16 with 3 bits is only 8% faster than BF16, while FP8 is 1.8×. Power savings are limited too. Decode (64 tokens) is bandwidth-bound; at 1000 W zeroing leaves its speed unchanged on uniform data, though on real Llama2-70B weights zeroed FP8 is still 2.6–3.0% faster at the same clock. FP8 zeroed to 0 bits saves 104 W on uniform data but only 61 W on real Llama2-70B weights, and truncating weights to 2 or 0 bits saves only 1.5% or 5.5% of power at an accuracy cost. Below 575 W these savings turn into 2–16% speedups, and prefill is power-bound even at 1000 W.

For weights and activations that directly set model quality and have no residual correction, zeroing turns directly into quality loss, in contrast with solvers such as HPL-MxP and SCF that correct errors. For GPUs without FP8 tensor cores, or pipelines that must stay in BF16, BF16 with 3–4 kept bits is a nearly lossless option; for FP8 deployments, zeroing saves only 1.5–5.5% of power in memory-bound phases, at an accuracy cost.

S8. Artifact Description

Hardware and power control. GH200: fourteen nodes, each with one NVIDIA GH200 480GB superchip (aarch64), 200 Gb/s InfiniBand, Slurm partition `compute`; module power caps are set with the cluster's `nv_power` tool only on exclusively allocated nodes; fans were left at the operator's setting. MI350X: two nodes with 8 MI350X OAM GPUs each, Slurm partition `gpu`; package power limits (PPT0) are set per GPU with `amd-smiset-oPPT0` and read back from sysfs; clocks for the locked-clock experiments with `amd-smiset-Lsclkmax`. Every job restores the default limit (and clock) on exit and records the cap in effect in each measurement window.

Software. GH200: NVHPC 26.5 (CUDA 13.2, cuBLAS 13.4, NCCL 2.29, HPC-X Open MPI 5), driver 615.71; ExaGeoStatCPP links cuBLAS 13.6. HPL is a CUDA port of rocHPL (branch `feat/cuda-port`) and HPL-MxP the `perf/h200-mantissa` branch of an HPL-MxP implementation [11]; Quantum ESPRESSO 7.6 [12]; GPU4PySCF 1.8.1 [13], [14]; MFEM 4.10.1 (CUDA, partial assembly); SW4 (`raja` branch, commit 52b51c9, RAJA 2026.07.1); LAMMPS 22 Jul 2025 Update 6 (KOKKOS, CUDA); GEMMul8 3.5.1; MLPerf Training Llama 2 70B LoRA with NVIDIA's v5.0 implementation. MI350X: ROCm 7.2.3 (rocBLAS, hipBLASLt, rocSOLVER, RCCL), amdgpu 6.16.13, Open MPI 5.0.7 with UCX; rocHPL branch `feat/adaptive-mixed-storage` (commit 37d0475), the same HPL-MxP source; GEMMul8 3.2.0 (commit 833e576); PySCF 2.14. Analysis uses Python with pandas, NumPy and Matplotlib.

Code and data. Table S5 lists where each part lives; the experiment logs in `gh200/` and `mi350/` record every job, including failed, cancelled and repeated ones, and every table and figure names its jobs and result files there.

Reproduction. Each experiment is one job that sets caps only on its own nodes, waits for the GPUs to cool before each run, interleaves variants in ABBA (or pairwise ABBA) order and writes power logs next to the program output; an analysis script then joins run windows with the power logs. For example, the GH200 crossover sweep of Section VII-A is `gh200/slurm/e1_gemm_tdp.sbatch` with the shapes, caps and methods passed as environment variables, summarized by `gh200/analysis/crossover.py`; the MI350X sweeps are `mi350/scripts/e1_gemm_tdp.sh` (GEMM, crossover, locked clocks via `e1b_fixedclk.sh`), `hpl_tdp.sh` (HPL and HPL-MxP), `gp_tdp.sh` and `mp2_tdp.sh`, submitted through `mi350/slurm/run.sbatch` and summarized by `mi350/analysis/e1_summary.py`, `hpl_summary.py`, `gp_summary.py` and `mp2_summary.py`; the model is `mi350/analysis/power_model.py`, evaluated with the idle P_0 by `model_eval.py` and on the two applications (locked-clock runs of `lockclk_apps.sh`) by `model_apps.py`. Intervals over independent units are `apps_units.py`; the HPL comparison under common residual thresholds is `hpl_threshold.py`; temperature adjustment is `temp_adjust.py`; the controlled thermal checks are `e1_gemm_tdp.sh` with a 4-minute warm-up and no cool-down, and `hpl_tdp.sh` with `PAIRWISE=1PRECOND=180`. The GEMMul8 size failure is reproduced in isolation by `mi350/apps/g8_repro.hip` (`g8_threshold.sh`), which compares GEMMul8's direct API with `hipblasDgemm` for $m = n$ around 65536.

References

[1] W. Dawson, K. Ozaki, J. Domke, and T. Nakajima, "Reducing numerical precision requirements in quantum chemistry calculations," *Journal of Chemical Theory and Computation*, vol. 20, no. 24, pp. 10 826–10 837, 2024.

[2] M. Fasi, N. J. Higham, M. Mikaitis, and S. Pranesh, "Numerical behavior of NVIDIA tensor cores," *PeerJ Computer Science*, vol. 7, p. e330, 2021.

[3] F. A. Khattak and M. Mikaitis, "Accurate models of NVIDIA tensor cores," arXiv preprint arXiv:2512.07004, 2025.

TABLE S5
WHERE EACH MECHANISM AND EXPERIMENT LIVES IN THE REPOSITORY.

Path	Content
`gh200/bench/`, `mi350/bench/`	mantissa GEMM microbenchmarks (cuBLAS; rocBLAS/hipBLASLt with GEMMul8 hook, transposes, exponent spread) and non-GEMM FP64 kernels
`gh200/shim/mantshim.cu`	cuBLAS interposer: rounding modes, cuBLAS emulation (fixed and automatic, bit-count log), FP32/3xTF32 replacement, exponent-span statistics
`rocHPL/src/blas/HPL_quantize_device.cpp`	HPL operand rounding (`HPL_QUANT_*_BITS`); the AMD build is branch `feat/adaptive-mixed-storage`
`rocHPL-MxP/src_mi350/hplmxp.h`	HPL-MxP zeroing on BF16 write-back (compile-time `MANT_BITS`), same source for both platforms
`apps/q-e/KS_Solvers/Davidson/`	in-program truncation and the tolerance-driven rule (`QE_MANT_ADAPT`)
`apps/pyscf-inputs/mp2_variants.py`, `mi350/apps/mp2_prep.py`, `mi350/apps/dfmp2.hip`	DF-MP2 variants on GH200; tensor preparation and HIP contraction on MI350X
`apps/geostat/gp_mle.py`, `mi350/apps/gp_chol.hip`	Gaussian-process likelihood and MLE (CUDA/cupy and HIP)
`mi350/apps/g8_repro.hip`	minimal reproducer of the GEMMul8 failure at $mn = 2^{32}$
`apps/patches/`	MFEM, SW4, LAMMPS and ExaGeoStatCPP changes
`gh200/slurm/`, `mi350/scripts/`, `mi350/tools/`	job scripts (cap sweeps with ABBA or pairwise ABBA order); MI350X power-cap tool and amdsmi sampler with energy accumulator
`gh200/analysis/`, `mi350/analysis/`	summaries, ΔW, intervals, crossover fit, power model (`power_model.py`), figures
`gh200/results/`, `mi350/results/`	summarized CSV files behind every table and figure

[4] J. Woo, S. Kim, and W. Y. Kim, "Dynamic precision approach for accelerating large-scale eigenvalue solvers in electronic structure calculations on graphics processing units," *Journal of Chemical Theory and Computation*, vol. 19, no. 5, pp. 1457–1465, 2023.

[5] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, and G. Bosilca, "Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion," in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 330–342.

[6] NVIDIA, "NVIDIA GH200 Grace Hopper superchip datasheet," Document 3773000, linked from https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/, Mar. 2025, accessed Oct. 5, 2026.

[7] P. Micikevicius, D. Stosic *et al.*, "FP8 formats for deep learning," arXiv preprint arXiv:2209.05433, 2022.

[8] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 37, 2015, pp. 1737–1746.

[9] J. Zhang, H. Huang, P. Zhang, J. Wei, J. Zhu, and J. Chen, "SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, arXiv:2411.10958.

[10] MLCommons, "MLPerf inference rules, section 8.1: Weight definition and quantization," https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc, accessed Oct. 5, 2026.

[11] J. Dongarra and P. Luszczek, "HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation," *The International Journal of High Performance Computing Applications*, vol. 40, no. 1, pp. 52–62, 2026, published online Sep. 25, 2025; arXiv:2509.19618.

[12] P. Giannozzi *et al.*, "Quantum ESPRESSO toward the exascale," *The Journal of Chemical Physics*, vol. 152, p. 154105, 2020.

[13] X. Wu *et al.*, "Enhancing GPU-acceleration in the Python-based simulations of chemistry frameworks," *WIREs Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.

[14] R. Li, Q. Sun, X. Zhang, and G. K.-L. Chan, "Introducing GPU acceleration into the Python-based simulations of chemistry framework," *The Journal of Physical Chemistry A*, vol. 129, no. 5, pp. 1459–1468, 2025.